Practical 4

Group 35:

Hayley Nel - u25101821
Keagan van Biljon - u23594412
Mohammed Lutchka -u25588304

Task 1

1. Domain Description and Problem Statement

A film production breaks a script down into a nested body of work. A Production is organised into Sequences (major narrative movements), each Sequence contains Scenes (individual locations or setups), and each Scene contains Shots - the actual camera setups a crew films. This is a part-whole hierarchy: a Sequence's total estimated hours are the sum of its Scenes, which are the sum of their Shots, and any level can be treated uniformly when reporting progress or estimating cost.

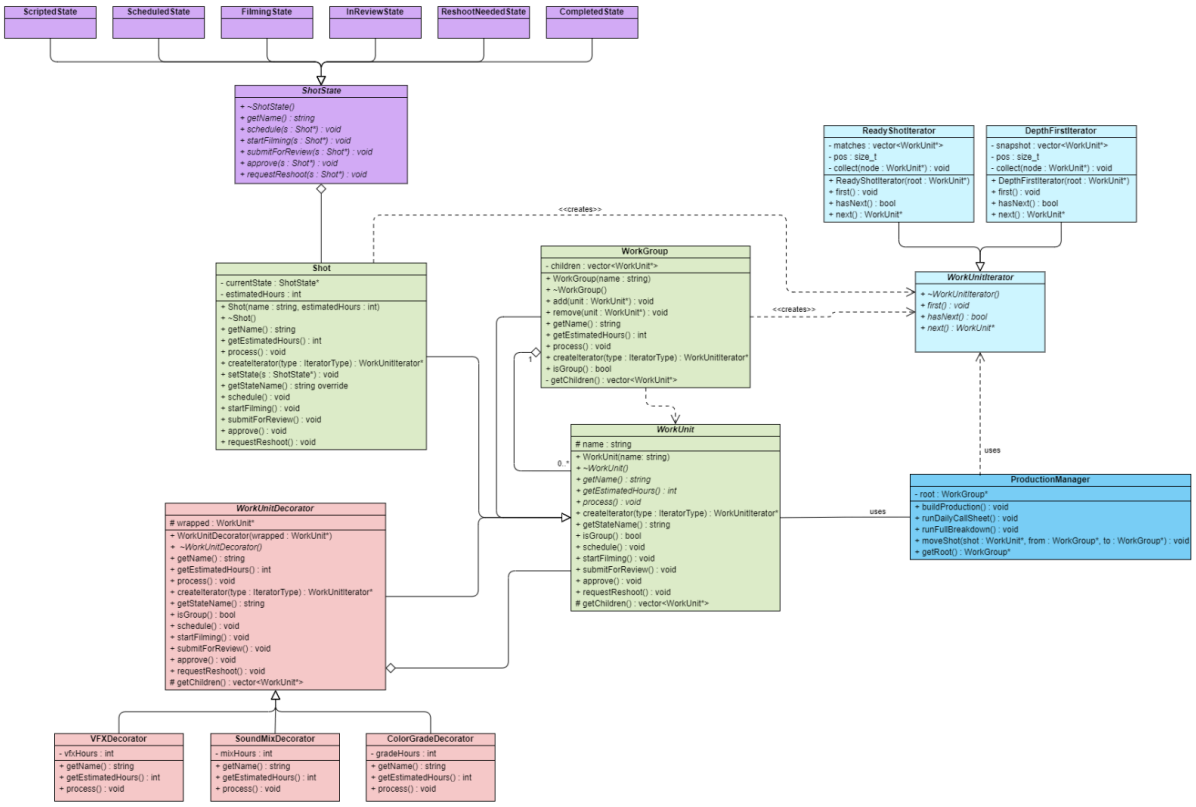
The production office needs to look at this hierarchy in different ways for different purposes. An assistant director building the full script breakdown needs to see everything, in order. A first AD building tomorrow's call sheet only wants the subset of shots that are currently ready to film - a different selection rule over the same structure, which must be able to run independently of, and at the same time as, other views.

Each individual Shot has a real production lifecycle: it starts out Scripted, gets Scheduled, moves to Filming, goes InReview once footage is in, and is either Approved/Completed or sent back as ReshootNeeded. What a shot does, and which actions are even valid, depends entirely on where it is in that lifecycle.

Finally, individual shots often need optional, stackable post-production work applied on top of the base shot - visual effects, a colour grade, a sound mix - without the shot becoming a different class of object. A heavily-decorated shot still needs to behave like any other shot everywhere else in the system.

The problem TaskForge solves: give a production office one coherent model of "the work" - where it lives in the hierarchy, what state it is in, what has been added to it - that supports multiple traversal needs, tracks realistic lifecycle behaviour, and lets responsibilities be added to shots at runtime, all without client code (schedulers, reports, call-sheet generators) needing to know about every concrete class or reach into internal containers.

2. UML Class Diagram



See class_uml.png to see diagram in more detail

3. GoF Participant Identification

Pattern	Abstraction / Interface	Concrete participants	Client
Iterator	WorkUnitIterator (Iterator)	DepthFirstIterator, ReadyShotIterator (ConcreteIterators); WorkGroup acts as the ConcreteAggregate via createIterator()	ProductionManager
Composite	WorkUnit (Component)	WorkGroup (Composite), Shot (Leaf)	ProductionManager, WorkUnitDecorator, iterators
State	ShotState (State)	ScriptedState, ScheduledState, FilmingState, InReviewState, ReshootNeededState, CompletedState (ConcreteStates)	Shot (Context)
Decorator	WorkUnit (Component); WorkUnitDecorator (Decorator)	VFXDecorator, ColorGradeDecorator, SoundMixDecorator (ConcreteDecorators)	ProductionManager, and the Composite/Iterator code, all via WorkUnit*

4. Inheritance, Associations, Multiplicities and Ownership

Every polymorphic base (WorkUnit, WorkUnitIterator, ShotState) declares a virtual destructor, so deleting through a base pointer is always safe. Ownership follows a simple rule throughout the design: whoever holds a pointer to construct a structure is responsible for destroying it.

Relationship	Type	Multiplicity	Ownership
WorkGroup -> WorkUnit (children)	Composition (aggregation of shared base)	1 WorkGroup to 0..* WorkUnit	WorkGroup owns its children; deletes them in its destructor
Shot -> ShotState (currentState)	Association (Context -> State)	1 to 1	Shot owns its current state object; replaced (old one deleted) on transition
WorkUnitDecorator -> WorkUnit (wrapped)	Composition	1 to 1	Decorator owns what it wraps; deleting the outermost decorator cascades through the whole stack
WorkGroup -> WorkUnitIterator	Dependency (<<creates>>)	N/A	Caller owns the returned iterator and is responsible for deleting it
WorkGroup / Shot / WorkUnitDecorator -> WorkUnit	Inheritance (realization of abstract interface)	N/A	N/A
ProductionManager -> WorkUnit / WorkUnitIterator	Dependency (uses)	1 root WorkGroup*	ProductionManager owns the root of the hierarchy only

5. Concrete Domain Classes

Beyond the pattern-required abstractions, the following concrete classes make the hierarchy and runtime behaviour non-trivial:

- WorkGroup - reused at every level of the hierarchy (a Production, its Sequences, and their Scenes are all WorkGroups), giving at least three levels of nesting below the client boundary and mixing individual Shots with nested groups at every level.
- Shot - the leaf work item; owns its current ShotState and delegates behaviour to it.
- ScriptedState, ScheduledState, FilmingState, InReviewState, ReshootNeededState, CompletedState - the concrete lifecycle stages, each defining which transitions are legal from that stage and rejecting the rest sensibly.
- DepthFirstIterator - full-structure traversal for the complete script breakdown / shot list; snapshots traversal order at creation time.

- ReadyShotIterator - selective traversal that yields only Shots currently ready to film, for the daily call sheet.
- VFXDecorator, ColorGradeDecorator, SoundMixDecorator - stackable post-production passes; each adds its own hours and its own step to process().
- ProductionManager - the client-facing driver that assembles the hierarchy, requests iterators, triggers state transitions and applies decorators, without ever touching WorkGroup's internal container directly.

6. Design and Ownership Rationale

Why a single WorkGroup class instead of separate Production / Sequence / Scene classes

The three grouping levels behave identically as composites - they hold children and aggregate estimated hours so a single reusable WorkGroup keeps the design as small as it can reasonably be while still supporting three genuine levels of nesting, as required. Domain-specific behaviour at a particular level (e.g. a Production-level report) can be added later as a thin subclass without disturbing the Composite structure.

Why iterators are created via a factory method rather than exposing the container

Client code (ProductionManager) is only ever given a WorkUnitIterator*, never the underlying std::vector<WorkUnit*>. This satisfies the rule against exposing a group's internal representation, and it is what allows two independent traversals (a full breakdown and a ready-shots call sheet) to exist over the same structure at the same time without interfering with each other.

Why DepthFirstIterator snapshots its order at construction

This is the team's traversal-modification policy: rather than iterating live over a structure that might change mid-traversal (a shot being moved between scenes for example), DepthFirstIterator captures the visiting order once, at creation. This is simple to reason about, safe against concurrent structural changes, and appropriate for a full breakdown, which is meant to represent a snapshot of the schedule at a point in time rather than a live view.

Why Decorator wraps WorkUnit rather than being specific to Shot

Wrapping the shared WorkUnit interface, rather than a Shot-specific one, means a decorated shot is still usable anywhere a plain WorkUnit is expected - inside a WorkGroup, through an iterator, or passed to ProductionManager - without any special-casing. It also means decorators can, in principle, apply to a WorkGroup as a whole in future without a design change.

Ownership policy

Ownership is deliberately hierarchical and single-parented: a WorkGroup owns its children, a WorkUnitDecorator owns what it wraps, and a Shot owns its current ShotState. Deleting the root of a structure - whether that root is a plain WorkGroup or the outermost layer of a decorator stack - cleanly cascades through the whole owned structure, which is what keeps the implementation free of leaks under Valgrind.

Task 2:

Your implementation must demonstrate all of the following somewhere in the running system:
• a genuine recursive part-whole hierarchy with meaningful leaf and grouping behaviour;
• traversal without exposing the hierarchy's internal representation to client code;
• at least two meaningful traversal behaviours over the same hierarchy;
• two independent traversals over the same runtime structure;
• state-dependent behaviour with valid and invalid transitions;
• at least two meaningful decorators that can be applied at runtime, including a case where decorators are stacked;
• sensible ownership and clean polymorphic destruction.

Task 3:

Traversal-Modification Policy

Both iterator implementations use a snapshot policy. When a DepthFirstIterator is created, it recursively traverses the current hierarchy and stores the traversal order internally. If the hierarchy changes after the iterator has been created, that existing iterator continues using the snapshot it captured at construction time.

For example, when Shot 1B is moved from Scene 1 to Scene 2, an already-created DepthFirstIterator does not rebuild its traversal order. The move removes the same WorkUnit* from one WorkGroup and adds it to another without destroying the object, so the pointers already stored in the iterator remain valid. A newly created DepthFirstIterator reflects the updated hierarchy and therefore sees Shot 1B in its new scene.

ReadyShotIterator follows the same snapshot principle. It determines which work units are ready when the iterator is created. If a shot changes lifecycle state afterwards, the existing iterator does not change its stored selection. A new ReadyShotIterator must be created to obtain an updated daily call sheet.

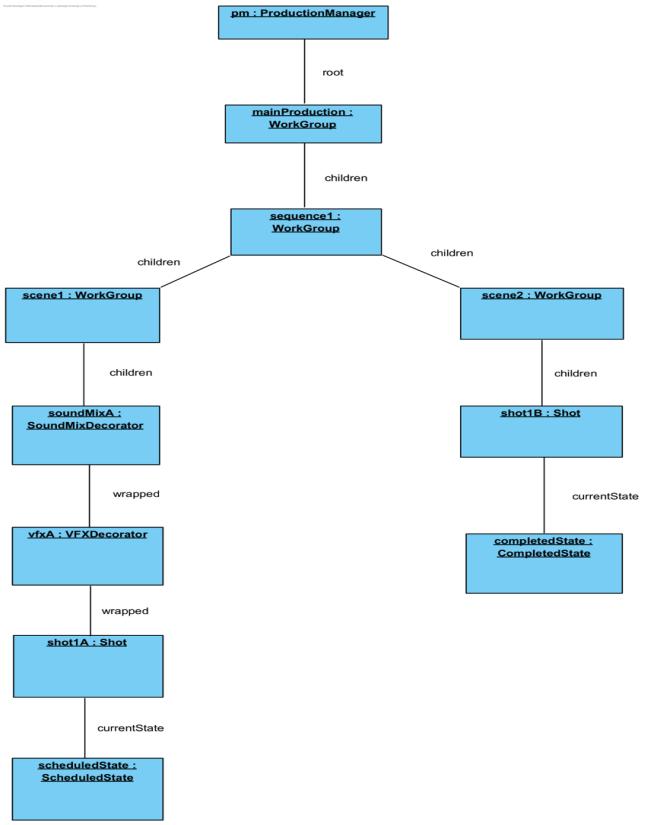
This policy was chosen because it makes traversal behaviour deterministic and prevents a traversal already in progress from changing unexpectedly when the underlying hierarchy or lifecycle state changes.

Create at least two non-trivial runtime scenarios in which several parts of the design collaborate. Across these scenarios, your program must include:
• traversal of nested objects;
• lifecycle/state-dependent behaviour;
• at least one decorated object participating in normal system behaviour;
• at least one runtime change to the structure, state, decoration, or a combination of these.
Your team must decide and document how an existing traversal behaves when the structure it refers to changes. Snapshot traversal, live traversal, invalidation, or another policy may be used, but the policy must be deliberate, safe and consistent with the implementation.

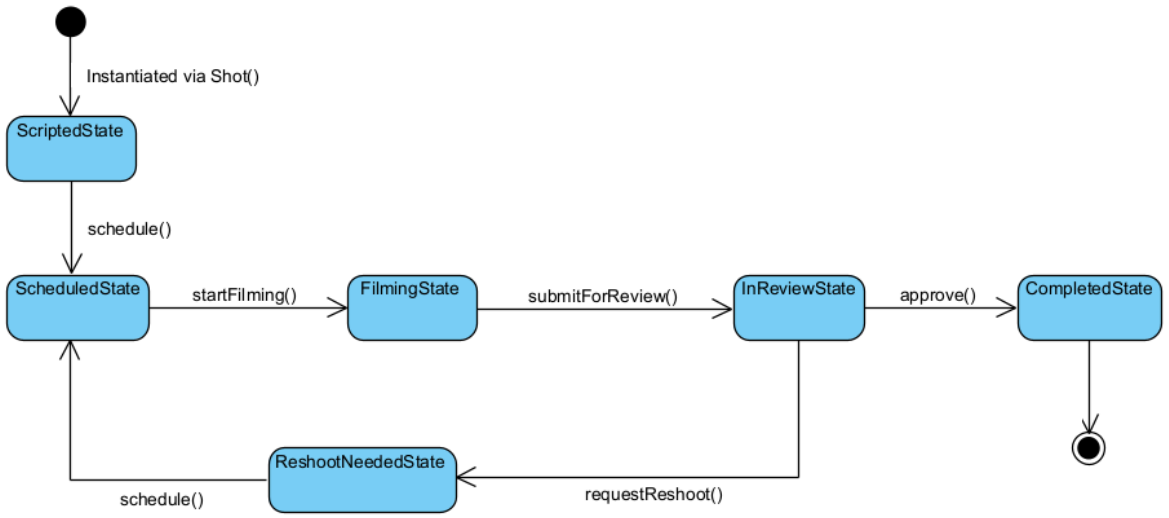
Task 4:

Submit a diagram portfolio that explains the design and the important workflows in your actual system.
In addition to the class diagram from Task 1, include:
• one UML object diagram showing a meaningful runtime portion of your nested hierarchy;
• one UML state diagram showing the lifecycle of an important stateful object;
• three substantial UML Activity Diagrams chosen to explain the most important workflows in your system.
The Activity Diagrams should not all tell the same story. Across the three diagrams, demonstrate appropriate use of actions, decisions, guards, merges and loops, as well as at least one fork/join, one called/composite activity, and one use of swimlanes. At least one Activity Diagram must make traversal visible as part of a larger workflow rather than hiding the entire traversal behind a single “process all” action. At least one should contain meaningful conditional behaviour related to the lifecycle or runtime configuration of an object. The third is your team’s choice and should showcase a distinctive workflow from your domain

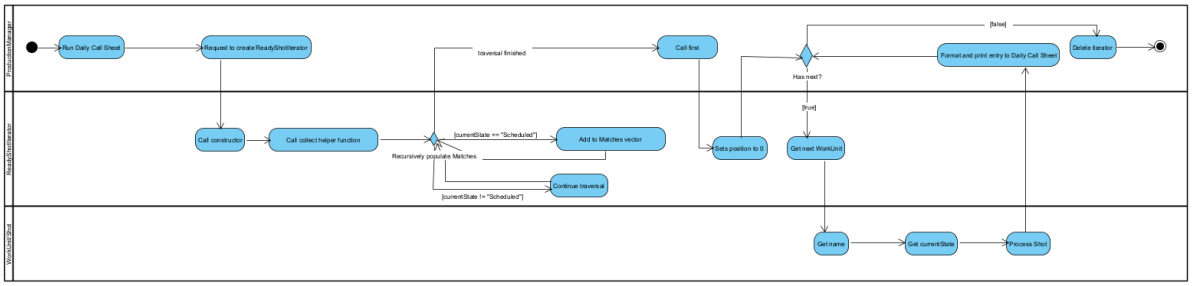
Object Diagram:**TaskForge runtime object diagram after Shot 1B is moved from Scene 1 to Scene 2.** The snapshot shows the nested Composite hierarchy, the stacked decorators applied to Shot 1A, and the current lifecycle states of Shot 1A and Shot 1B.



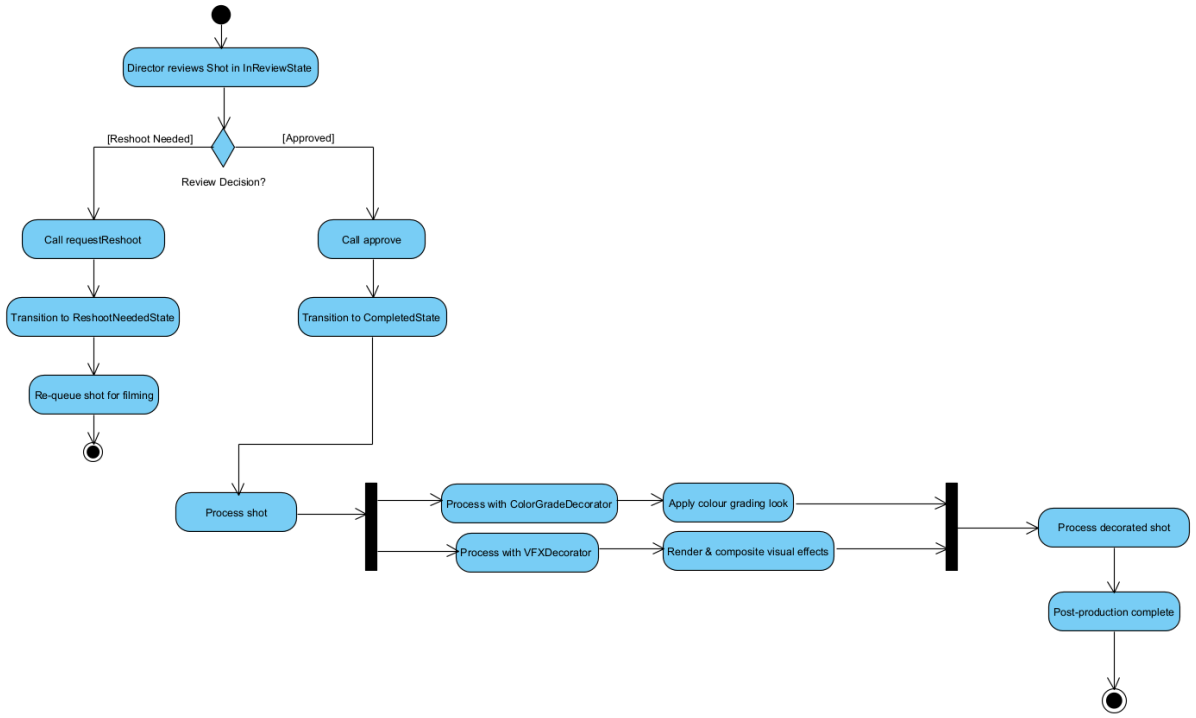
State Diagram: Shot Lifecycle



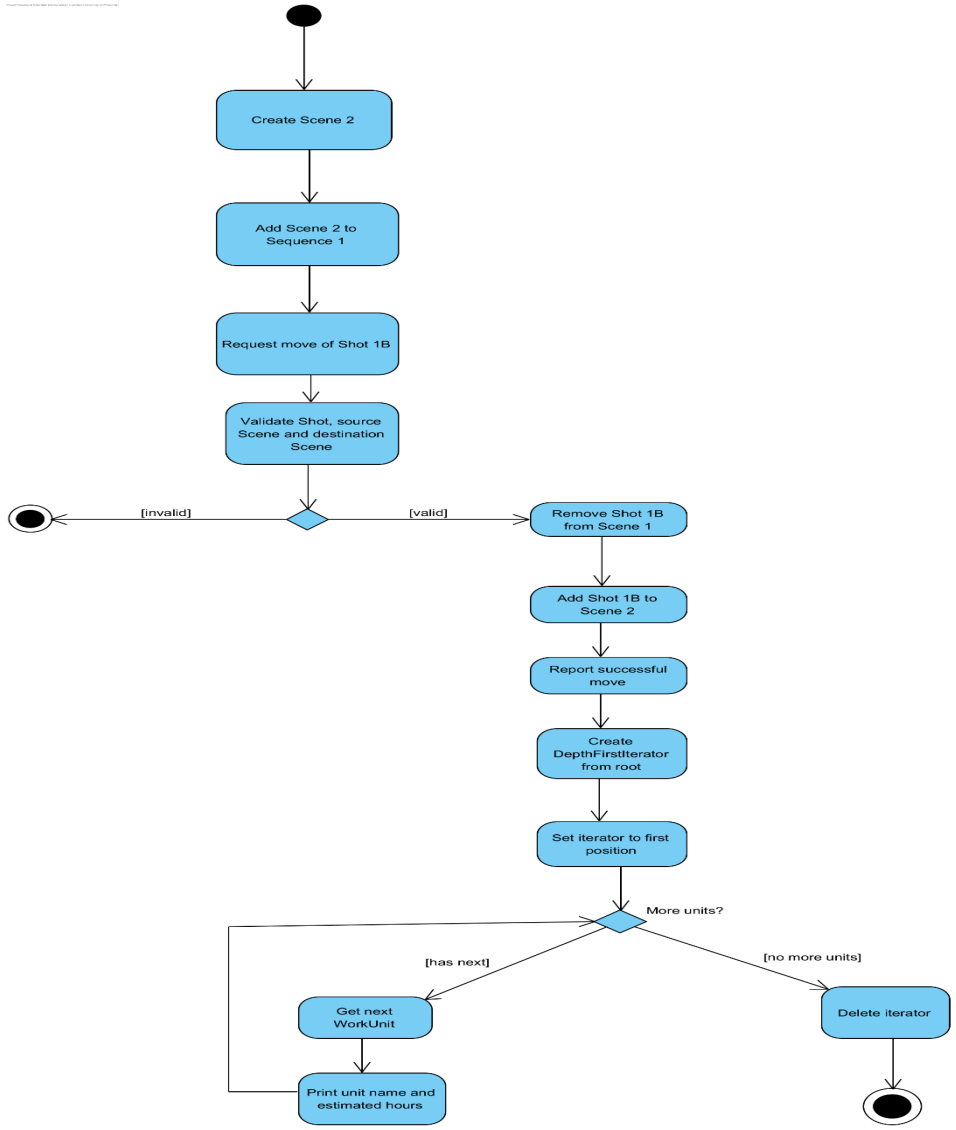
Activity Diagram 1: Daily Call Sheet Generation (ReadyShotIterator Traversal)



Activity Diagram 2: Shot Approval & Post-Production Pipeline



Activity Diagram 3:Move Shot and Generate Updated Production Breakdown



Task 5:

Valgrind Memory Check

```
========== Done ==========
==1==
==1== HEAP SUMMARY:
==1==     in use at exit: 0 bytes in 0 blocks
==1==   total heap usage: 107 allocs, 107 frees, 14
4,503 bytes allocated
==1==
==1== All heap blocks were freed -- no leaks are po
ssible
==1==
==1== For lists of detected and suppressed errors,
rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (supp
ressed: 0 from 0)
```

Valgrind was run on the final taskforge executable inside the Docker container using --leak-check=full --show-leak-kinds=all. The final report showed 0 bytes in 0 blocks still allocated at exit, confirmed that all heap blocks were freed, and reported 0 errors from 0 contexts. This indicates that the final implementation contains no detected memory leaks or invalid memory operations.

Breakpoint hit

```
-- Moving Shot 1B from Scene 1 into new Scene 2

Breakpoint 1, ProductionManager::moveShot (
    this=0x7ffe0953b2a0, shot=0x628511e8a820,
    from=0x628511e8a790, to=0x628511e8a9d0)
    at ProductionManager.cpp:81
81      void ProductionManager::moveShot(WorkUnit* shot, WorkGroup* from, W
orkGroup* to) {
(gdb)
```

Inspecting program state

```
    at ProductionManager.cpp:81
81      void ProductionManager::moveShot(WorkU
(gdb) set print pretty on
(gdb) print shot
$1 = (WorkUnit *) 0x628511e8a820
(gdb) print from
$2 = (WorkGroup *) 0x628511e8a790
(gdb) print to
$3 = (WorkGroup *) 0x628511e8a9d0
(gdb) print shot->getName()
$4 = "Shot 1B"
(gdb) print from->getName()
$5 = "Scene 1"
(gdb) print to->getName()
$6 = "Scene 2 (reshoot pickup)"
(gdb) print from->children.size()
$7 = 2
(gdb) print to->children.size()
$8 = 0
(gdb)
```

Stepping through the move

```
$8 = 0
(gdb) list
76          }
77
78          delete it;
79      }
80
81      void ProductionManager::moveShot(WorkUnit* shot, WorkGroup* from, WorkGroup* to) {
82          if (!shot || !from || !to){
83              return;
84          }
85
(gdb) next
82          if (!shot || !from || !to){
(gdb) next
86          from->remove(shot);
(gdb) next
87          to->add(shot);
(gdb) print from->children.size()
$9 = 1
(gdb) next
89          std::cout << "Successfully moved '" << shot->getName() << "' from '" << from->g
etName() << "' to '" << to->getName() << "'" << std::endl;
(gdb) print to->children.size()
$10 = 1
```

Use the debugging tools introduced in the module as part of development, not only after the program is finished.
Your PDF must contain a short engineering investigation that includes:
• evidence of using GDB on a meaningful part of your program, including breakpoints/stepping and inspection of relevant program state;
• one genuine bug encountered during development, with its symptom, cause, debugging evidence and correction;
• Valgrind evidence for the final application. The final implementation must not contain definitely-lost memory originating from your own code.

Task 6:

The repository is both the development environment and evidence of the team's engineering process. Your repository must contain at least:
• all source/header files and main.cpp;
• a working Makefile;
• a working Dockerfile;
The Docker environment must provide the tools needed to compile, run and investigate the system, including g++, make, gdb and valgrind.
• README.md;
The README.md must contain sufficient commands for a marker to reproduce the build and execute the program, GDB and Valgrind without installing project-specific dependencies directly on the host machine.
• a docs/ directory containing the submitted design material or exported diagrams.
Git history must show genuine work by all three members across the development period. Use meaningful commit messages.
Branches, pull requests, issues and reviews are encouraged where useful, but the important requirement is a credible collaborative history rather than a particular Git workflow.
Include a short reflection in the PDF explaining how work was divided and integrated, with references to a few commits, pull requests, issues or other repository activities that demonstrate collaboration.
As a team we noted down all tasks and then we each picked equal amounts of work to be done. It was then integrated first through design. Keagen worked on the UML. We then moved onto the implementation of the design. Mohammed focused on State and Composite, Hayley on Iterator and Keagen on Decorator. This can be seen to be done collaboratively via the commits on Github such as "Add ShotState lifecycle abstraction" by Mohammed, "Iterator: TODO: Populate matches in constructor" by Hayley and "Added full decorator" by Keagen. These commits were made across the development period as we worked together. Our collaborative history can also be seen through our workflow. We worked in separate branches for each pattern to avoid merge conflicts. This strategy worked well, making it easy to then merge patterns together when we put our work together. This merging can be seen by Keagen's numerous commits: "Merge pull request # .."

Github repo link: https://github.com/Hibikus/COS214_Practical4

Task 7:

Prepare a coherent demonstration of approximately five minutes.
Do not structure the program as a menu that simply runs one pattern example after another.
The demonstration should tell a believable story in your chosen domain and naturally expose the important design decisions.
During the demonstration, a tutor may ask any team member to:
• identify where a GoF participant appears in the code;
• explain an ownership or lifetime decision;
• explain how traversal state is maintained and what happens after a runtime modification;
• trace an Activity Diagram action or decision to the implementation;

• explain a state transition or decorated behaviour;
• run or discuss GDB and Valgrind evidence;
• build/run the program in Docker; or • discuss the team's Git history.
All three team members are responsible for understanding the complete submission.

Team Contribution:

Hayley Nel
- Iterator implementation
- State Diagram
- Activity Diagram 1
- Activity Diagram 2
- Task 7

Keagan van Biljon
- UML
- Task 1
- Decorator Implementation
- Fitchfork and demo Main

Mohammed Lutchka
- State Implementation
- Composite Implementation
- Task 3
- Task 5
- Docker
- Object Diagram
- Activity Diagram 3

Checklist:

The PDF should contain, in a sensible order:

• system description and team details;
• completed UML class diagram and GoF participant mapping;
• concise design/ownership rationale;
• object diagram and state diagram;
• the three Activity Diagrams;
• the team's traversal-modification policy and any other important design decisions;
• GDB bug investigation and Valgrind evidence;
• GitHub repository link and workflow reflection;
• a short team contribution statement.